AURA Project Explanation

1. Overview of AURA

AURA stands for Autonomous University Response and Action. It is a Java desktop application designed for campus issue management and suggestion handling. The project aims to replace hidden and scattered complaint channels such as WhatsApp groups, informal reporting methods, and paper-based suggestions with a structured and transparent digital system.

The main purpose of AURA is to allow students to raise problems or suggestions safely and anonymously while enabling administrators to manage, filter, prioritize, track, and resolve them in an organized manner. The system combines issue reporting, anonymous tracking, community upvoting, status monitoring, and administrative resolution into a single platform.

AURA is designed for academic institutions, especially universities and colleges, where issues such as broken infrastructure, academic problems, hostel difficulties, electrical faults, and general campus concerns must be reported systematically and resolved efficiently.

2. System Architecture of AURA

AURA follows a layered architecture that clearly separates different responsibilities. The architecture is based on four layers:

1. Presentation Layer

2. Service Layer

3. Data Access Layer

4. Database Layer

The main design rule is:

UI → Service → DAO → Database

This ensures that the graphical interface does not directly interact with the database, which improves organization, security, and maintainability.

3. Class Diagram Explanation

The class diagram is the structural blueprint of the project. It explains the main classes, their properties, and their relationships.

3.1 User

The `User` class is the base model for both students and administrators. It contains details such as:

This class is central to authentication and access control.

3.2 Student

The `Student` class represents a student user. A student can:

This class inherits the general properties of the `User` class.

3.3 Admin

The `Admin` class represents an administrative user. An admin can:

The admin role is different from the student role because the admin has authority to manage the issue workflow.

3.4 Submission

The `Submission` class is the core entity of AURA. It represents a single issue or suggestion raised by a student. Its fields include:

This object is the central record of every report submitted into the system.

3.5 Issue and Suggestion

The `Issue` and `Suggestion` classes inherit from `Submission`. This reflects the fact that both are categories of reports but may require different handling.

This use of inheritance helps model the domain cleanly.

3.6 SubmissionHistory

This class records the lifecycle history of a submission. It stores:

This allows the system to maintain traceability. For example, the system can track when a ticket moved from Pending to Assigned to In Progress and finally to Resolved.

3.7 ResolutionNote

This class stores the official note added by an administrator when resolving or rejecting an issue. It helps keep a record of the final action taken.

3.8 StudentReceipt

This class stores the private relationship between a student and their submitted issue. It is crucial for anonymity. The student can find their own submitted issues through this mapping without exposing their identity publicly.

3.9 SubmissionHype

This class models the vote or hype action. It stores:

It allows students to support issues that are important to them, which generates the trending mechanism in the system.

3.10 Enums

The project uses enumerations for structured values:

These help maintain consistency and prevent invalid values.

4. UML Use Case Diagram Explanation

The UML Use Case diagram describes how different actors interact with the system.

4.1 Actors

The two major actors are:

4.2 Student Use Cases

A student can:

This shows that the student is mainly involved in creating and tracking campus concerns.

4.3 Admin Use Cases

An admin can:

This shows that the admin is responsible for proper issue management and resolution flow.

4.4 Why it matters

The use case diagram helps the team define the software's functional scope. It explains what actions the user can perform and what the system must support. It also helps communicate the project clearly during presentations and viva interviews.

5. Flow Chart Explanation

The flow chart describes how the application works step by step.

5.1 Start

The process begins when the user starts the application and reaches the login screen.

5.2 Login Validation

The user enters credentials. The system verifies them against the database. If valid, the system determines whether the user is a student or admin.

5.3 Student Flow

If the user is a student, the system opens the student dashboard. The student can:

Once the issue is submitted, it is saved in the public submission table, and the student gets a private receipt link for tracking.

5.4 Admin Flow

If the user is an admin, the system opens the admin dashboard. The admin reviews all open tickets and decides the next action. The admin can:

5.5 Closure

After the final action, the submission is marked as Resolved or Rejected, and the corresponding history and note records are created. This closes the workflow in a transparent and auditable way.

The flow chart is important because it explains how the real process works inside the application.

6. Database Design Explanation

The database is the core persistence layer of AURA. It organizes the application's data into a relational structure.

6.1 Users Table

This table stores all users of the application, including students and administrators. It keeps:

The design ensures only institutional emails are accepted.

6.2 Submissions Table

This table stores the public issue or suggestion information. It does not store the submitting student ID. This is important for anonymity.

The submission row contains fields like:

6.3 Student Submission Receipts Table

This table links a student to their submission. It acts as a private map that allows the student to track their own reports without exposing their identity to the rest of the system.

This is one of the strongest design decisions of the project.

6.4 Submission Hype Table

This table stores each upvote/hype action. It ensures that a student can hype a submission only once. This helps the trending mechanism and prevents spam or duplicate support.

6.5 Resolution Notes Table

This table stores the final feedback or action taken by the admin regarding a submission.

6.6 Submission History Table

This table records every change in the status of a ticket. It gives the project a complete audit trail and transparency.

6.7 Why the database design matters

The database is designed to support security, privacy, and operational workflow. It helps maintain data integrity and makes the application suitable for real campus use.

7. Wireframe Model Explanation

The wireframe model describes the visual design of the application screens.

7.1 Login Screen

The login screen includes fields for:

This is the first interaction point for the user.

7.2 Student Dashboard

This screen contains the main student features:

7.3 Submission Form Page

This page allows students to add a new report. It includes the necessary fields to describe the problem clearly and categorize it correctly.

7.4 Admin Dashboard

This screen contains the admin queue, filtered by issue status and category. It helps the admin prioritize and manage tasks effectively.

7.5 Issue Detail / Resolution Page

This screen shows the issue details and lets the admin update the status and add a resolution note. It is a crucial part of the admin workflow.

7.6 Importance of wireframes

Wireframes help translate the technical design into a user-friendly layout. They ensure that the system is not only functional but also understandable and easy to use.

8. Major Classes and Their Importance in the Entire Project

Main classes and their roles:

These classes together form the complete application architecture.

9. Why the architecture is good

The architecture of AURA is strong because it follows good software engineering practices:

This makes AURA a realistic and professional academic project.

10. Final Summary

AURA is a smart campus issue reporting system designed to improve communication between students and administrators. It uses a layered architecture, clear class structure, proper database design, use case modeling, workflow flowchart, and user-friendly wireframes to provide a complete and practical software solution.

The most significant strengths of AURA are:

Together, these features make AURA not only academically valuable but also practically useful for real university operations.

Prepared for academic explanation of the AURA project architecture and design.

- This layer contains the user interface components.
- It includes login screens, student dashboards, admin dashboards, and submission forms.
- The user interacts with this layer directly.
- This layer contains business logic and validation rules.
- It handles authentication, submission processing, status updates, hype logic, and user role checks.
- This layer is responsible for interacting with the database.
- It uses JDBC and DAO classes to perform database operations safely.
- The database stores user information, submissions, hype records, notes, and history.

- The database used in this project is PostgreSQL through Supabase.
- userId
- name
- email
- passwordHash
- role
- submit issues
- submit suggestions
- view their own submissions
- check status of submissions
- hype relevant issues
- view all submissions
- review ticket details
- assign issues
- update status
- add resolution notes
- generate reports
- title
- description
- type
- category
- location
- priority
- status
- photoUrl
- hypeCount
- createdAt
- `Issue` is used for complaints or problems in campus facilities
- `Suggestion` is used for proposals or improvements
- submissionId
- oldStatus
- newStatus
- changedBy
- changedAt
- submissionId
- studentId
- createdAt
- Role

- SubmissionType
- Category
- Priority
- SubmissionStatus
- Student
- Admin
- register and log in
- submit issue
- submit suggestion
- view own submissions
- check submission status
- hype a report
- read resolution notes
- log in
- review all submissions
- filter issues
- assign tasks
- update issue status
- add resolution notes
- generate reports
- submit a new issue
- see previous submissions
- view personal tracking information
- hype a report
- see status updates
- view issue details
- assign issue
- change status
- add resolution note
- close the issue
- user ID
- name
- email
- password hash
- role
- creation time
- title
- description

- category
- type
- status
- location
- priority
- photo URL
- created time
- institutional email
- password
- login button
- Google sign-in option
- submit issue
- view personal submissions
- check issue status
- check trending issues
- access account actions
- `Main`: application start point
- `LoginFrame`: authentication screen
- `AuthService`: login and session logic
- `UserDAO`: user database operations
- `SubmissionDAO`: submission and status persistence
- `StudentReceiptDAO`: private ownership mapping
- `HypeService`: hype and trending logic
- `SubmissionHypeDAO`: vote persistence
- `DatabaseSubmissionService`: central submission service implementation
- `StudentDashboardFrame`: student view
- `AdminDashboardFrame`: admin control panel
- `SubmissionFormPanel`: issue/suggestion form
- `ValidationUtil`: validation logic
- `PasswordUtil`: password hashing and verification
- `SessionContext`: current user session
- Clear separation of responsibilities
- Secure and structured access control
- Privacy protection through data separation
- Database integrity using constraints and keys
- Maintainable layered design
- Clear distinction between student and admin functions
- Auditable ticket lifecycle system

- User-friendly desktop interface
- anonymity of student reports
- systematic issue processing
- admin tracking and resolution
- database-based integrity
- role-based access control
- transparent status lifecycle
- trending and prioritization of issues